Bouncer: Competence Auditing for Set-Local Learned Microarchitectural Controllers

Anonymous Submission
Paper #— MLSys Submission

***Abstract*—Learned controllers—reinforcement-learning prefetchers, imitation-trained replacement policies, RL memory schedulers, neural branch predictors—are entering the datapath because they beat hand-tuned heuristics *on average*. But their advantage is workload- and phase-dependent and can silently invert under distribution shift, and a co-located adversary that knows the design can *steer* a shared controller toward its decision boundary while looking innocuous on aggregate performance counters. We present Bouncer, a runtime monitor and trust gate that—for a declared domain audited at finite resolution with its secret intact—bounds the worst-case *competence* cost (in the controller's own reward) of any *set-local* controller—one whose decision and its reward fall on the same dueling set, canonically cache *replacement*—to that of a known-safe fallback heuristic, under adaptive mimicry and benign drift, with no POMDP/belief-state machinery and, by construction, off the cache-access critical path (the confirmer is off-path and epoch-grained; the resulting no-added-latency property is analytical, with RTL timing measurement left to future work); a sub-resolution *slow-bleed* steering attack is an explicitly named open vulnerability. *Set-locality is the enabling condition*: it is exactly what lets a secret per-set audit attribute reward to the right policy, and it cleanly separates the controllers Bouncer faithfully audits (PC-indexed predictors, and cache replacement under a (near-)stateless reward) from the cautionary boundary case (prefetching, whose benefit is de-localized). Bouncer reduces "is the learned controller still trustworthy?" to a single scalar—a competence advantage Δ_W over the fallback—estimated *without counterfactuals* by repurposing the set-dueling substrate already in silicon, but with a twist: the run-learned / run-fallback set assignment is drawn from a hardware secret and reseeded each epoch. Cheap always-on tripwires gate an expensive competence confirmer whose one-sided CUSUM feeds a four-state trust FSM. We prove a bounded-regret safety floor (over the union of audited declared domains, in the controller's own reward, Bouncer is never more than $N_{ep}{\cdot}D{\cdot}r_{\max} + \phi_P T_{att} r_{\max} + \alpha T c_{sw}$ worse than the fallback: a per-episode detection transient, an audit-exposure term that grows only at the dueling fraction of attack duration, and a false-alarm term) and a *per-domain* mimicry-resistance property that follows exactly from the secrecy of the sampler—scoped to a declared region at finite resolution with the secret intact, not general any-region security. In a faithful, fully-released *synthetic competence harness* the estimator tracks ground truth at R^2=0.997, the steady-state safety floor holds to within 0.63% of the fallback under attack, detection costs one window, the clean-workload tax is 0.35%, and the auditor fits in $\sim$336 bytes (an *analytical* budget: $\sim$0.13% of a 256 KB SRAM; the confirmer is off-path and epoch-grained, so it adds no latency to the cache-access critical path). The headline result (over 50 seeds): under an adaptive mimicry adversary *within a declared, secret-intact domain*, an input-OOD monitor detects *none* of the attacks ($0/50$, Wilson 95% upper bound 0.07) while Bouncer detects *all* ($50/50$, lower bound 0.93)—a robustness purchased entirely by the secrecy of the dueling sets, which collapses both as the assignment leaks and against a sub-resolution redistributive (slow-bleed) adversary outside the audited domain. We probe the set-locality boundary on real ChampSim deployments on SPEC CPU2017. As an L1D prefetcher module it bounds a corrupted prefetcher to the prefetch-off floor but, as a non-set-local controller, over-gates a genuinely-helpful prefetcher under a per-set cache-hit proxy—and swapping to the controller's own reward does *not* fix it, because a prefetcher's reward cannot be localized to a set. As an LLC *replacement* module it *is* set-local, yet exposes a second, subtler boundary: a cache's reward is *stateful* (path-dependent), so the secret per-epoch reseed that buys mimicry resistance *confounds* the per-window competence estimate (fixed leaders resolve it; the prior "$\hat{\Delta} \approx 0$" was in fact a missing-callback software artifact, now fixed). Set-locality is therefore necessary but not sufficient: a secret, reseeded audit also needs a (near-)stateless reward. The mechanism's quantitative guarantees are validated in a fully-released synthetic harness whose i.i.d. per-decision reward meets exactly that condition; the ChampSim integration is a real-simulator *boundary* study, not a clean-case validation or a SPEC sweep.**

I. Introduction

A decade of "ML for systems" has put learned controllers next to, and increasingly *inside*, the datapath: Pythia learns a prefetch policy online with reinforcement learning [1]; Hawkeye and Glider learn cache replacement by imitating Belady [2], [3]; RL has been applied to memory scheduling [4]; and perceptron/TAGE predictors dominate branch prediction [5], [6]. These designs win because, *on the distribution they were validated on*, they beat a simple baseline. That qualifier is the whole problem. The advantage of a learned controller C over a safe heuristic π_0 is a function of the workload and execution phase, and it can go to zero—or negative—in two ways that current deployments do not detect:

- **Benign drift.** A phase change or an unseen application moves the controller off its validation distribution D_{val} into a silent QoS regression. The counters still look healthy in aggregate; the learned policy is simply now worse than the heuristic it replaced.
- **Adversarial steering.** A co-located tenant who shares C and knows its design (Kerckhoffs) crafts memory accesses that drive C toward its decision boundary—maximizing its loss while keeping aggregate counters in-distribution. This is the regime of prefetcher- and predictor-steering attacks [7], [8] and of contention/occupancy channels [9], [10].

The design rests on one asymmetry. A learned controller has *unbounded upside and unbounded downside*; a vetted heuristic (π_0 = next-line/off, RRIP/LRU, FR-FCFS, gshare) has *modest* *upside but a bounded, attack-resistant downside*. We would like to keep the upside while capping the downside at the heuristic's floor—over the audited declared domains and in the controller's own reward metric. That is a hedging problem, and Bouncer is the constructive hedge, one whose cap we show has an explicit sub-resolution coverage hole.

a) Why not just monitor the inputs?: The natural reflex is an OOD/drift detector on the controller's input features [11], [12], [13]. This is necessary but neither sufficient nor safe: an adversary can hold the input *marginals* in-distribution while degrading control quality (the mimicry attack), and an input monitor is blind to it by construction. Bouncer instead measures *realized competence*—the reward the controller actually earns relative to the fallback—a signal an attacker cannot fake, within a declared secret-intact domain, without actually making the controller good (in that reward metric). The certificate is over the controller's own reward, not end utility, which it tracks only insofar as the reward proxies utility (Section XI-J).

b) The set-locality condition (the organizing insight).: A secret per-set audit can only certify a controller whose *reward is attributable to the same dueling set as its decision*—we call such a controller *set-local*. Cache *replacement* is the canonical set-local controller: the eviction decision on set s is rewarded by the hit/miss outcome *on set s*, so the dueling estimate $\hat{\Delta}$ is a clean, localized competence signal. PC-indexed predictors (branch, confidence) are likewise local. *Prefetching is the cautionary boundary*: a prefetch triggered on set s fetches a *different* set s', so its benefit is de-localized and no per-set reward captures it—a faithful reward cannot be dualized, which we confirm in real ChampSim (Section XI-J). Set-locality thus *characterizes when competence auditing is faithful*, and reframes Bouncer as a clean mechanism for the set-local class (replacement first), with prefetching and the coverage hole as its honest boundaries.

c) Contributions.:

1) **The set-locality characterization** (Section II): the condition—decision and reward share a dueling set—that determines *when* secret-set-dueling competence auditing is faithful, separating the clean class (replacement, PC-indexed predictors) from the de-localized boundary (prefetching), which we confirm in real ChampSim. We further show it is *necessary but not sufficient*: a secret, *reseeded* audit also needs a (near-)*stateless* reward (Remark 1), because reseeding—the source of mimicry resistance—erases the policy-specific state a path-dependent reward (a real cache) needs to be measured.
2) **A scalar off-policy condition** (Section II) that unifies benign drift and adversarial steering: the competence advantage Δ_W falling below a trust threshold τ.
3) **Randomized-secret set-dueling** (Section V): a counterfactual- free estimator $\hat{\Delta} = \bar{r}_L - \bar{r}_F$ that repurposes the DIP/DRRIP sampling substrate [14], [15] from policy *selection* to policy *trust*, with the set assignment kept secret.
4) **Two guarantees with worked constants** (Section VII): a three-term bounded-regret safety floor (Lemma 1: detection, audit-exposure, and false-alarm terms, over the union of audited declared domains in the controller's own reward) tied directly to the CUSUM average-run-length theory [16], [17], [18], and a *per-domain* mimicry-resistance proposition (Proposition 1)—scoped to a declared region audited at resolution $\delta_R^{\min}(B)$ with an intact secret, with the sub-resolution slow-bleed attack named as an open vulnerability—whose strength is exactly the secrecy of the sampler.
5) **A two-tier microarchitecture** (Sections IV and XII) whose analytical budget is ~ 336 bytes and which, because the confirmer is sampled, off-path, and epoch-grained, is designed to add no latency to the cache-access critical path (an analytical argument; RTL timing measurement is future work, Section XII).
6) **A faithful, released evaluation** (Section XI) validating the mechanism's quantitative claims in a synthetic competence harness (with multi-seed confidence intervals, sensitivity sweeps, and adaptive timing adversaries), *plus* a real **ChampSim L1D prefetcher module** run on SPEC CPU2017 that bounds a corrupted prefetcher to the safe floor but over-gates a genuinely-helpful one (11% tax on roms), candidly showing the auditor is only as good as the competence reward it is given—and that the reward's *localizability*, not just its fidelity, is decisive.

We are explicit (Sections X and XIV) that the controlled quantitative claims come from the synthetic harness—whose only load-bearing assumptions, bounded reward and a partitionable resource, are exactly those the guarantees rest on—while the ChampSim integration demonstrates the mechanism in a real simulator on a 3-trace SPEC suite rather than a full sweep.

II. Formal setting

A learned controller C is invoked at decision points $t = 1, 2, \ldots$. At each it observes microarchitectural state $x_t \in \mathbb{R}^d$, emits action $a_t \in \mathcal{A}$, and after a short horizon the environment yields a *bounded* reward $r_t \in [0, r_{\max}]$ (prefetch hit×timeliness, cache hit, −latency, branch correct). C was trained/validated on a distribution D_{val} of (x, a, r). A fixed safe fallback π_0 has a known, attack- resistant competence floor.

Definition 1 (Competence advantage)**.** *Over a window W,* $\Delta_W = \frac{1}{|W|} \sum_{t \in W} \left(r_t^C - r_t^{\pi_0} \right)$.

$r_t^{\pi_0}$ is counterfactual—one cannot run both policies on the same decision—which is the central estimation challenge Section V solves.

Definition 2 (Off-policy condition)**.** *The system is* off-policy *at W iff $\Delta_W < \tau$ for a trust threshold $\tau \geq 0$. τ=0 means "learned no longer beats the floor"; τ>0 gives margin and hysteresis.*

This one scalar unifies the two threats: an adversarial push toward C's decision boundary and a benign distribution shift both manifest as Δ_W falling. The auditor detects the condition; a lightweight triage (Section VIII) guesses the cause to pick the response.

Definition 3 (Set-locality)**.** *Let the resource be partitioned into dueling sets $\{s\}$. A controller is* set-local *if the reward r_t for a decision made on set s is realized on set s (so it can be attributed to s's pool). Then the per-pool means $\bar{r}_L, \bar{r}_F$ are unbiased estimates of each policy's competence on the sampled sets, and $\hat{\Delta} = \bar{r}_L - \bar{r}_F$ is a faithful competence estimate.*

Set-locality is the enabling condition for the entire mechanism: it is precisely what makes $\hat{\Delta}$ (Section V) a clean signal rather than a mis-attributed one. Cache *replacement* satisfies the spatial-attribution requirement exactly (the eviction on s is rewarded by hits/misses on s); PC-indexed predictors satisfy it too—though, as Remark 1 and Section XI-J show, replacement's reward is *stateful*, so a secret reseeded audit does not recover a faithful $\hat{\Delta}$ there at per-window grain; *prefetch* violates it (a prefetch on s benefits a different set s'), and we show in Section XI-J that no per-set reward—not even the controller's own— recovers a faithful $\hat{\Delta}$ for it. We therefore present Bouncer as a mechanism for the *set-local* class, with replacement as the canonical instance and prefetching as the characterized boundary.

Remark 1 (Set-locality is necessary, not sufficient—a stateless-reward condition)**.** *The unbiasedness in Definition 3 holds when the reward r_t depends on the policy run on s but* not *on a long policy-specific* history *of s. If the reward is* stateful*—e.g. a cache hit on s depends on which blocks prior evictions on s retained—then a* secret, reseeded *sampler breaks it: reseeding the leader assignment each epoch (the source of mimicry resistance, Proposition 1) prevents any set from running one policy long enough to accumulate its policy-specific state, so $\bar{r}_L \to \bar{r}_F$ and $\hat{\Delta}$ collapses to noise even though the controller is set-local. Our synthetic harness's i.i.d. per-decision reward (A1) is stateless and so unaffected; real cache replacement is set-local but stateful, and Section XI-J measures this secrecy/measurability tension directly (fixed leaders recover the gap a reseeded audit cannot). A secret, reseeded competence audit thus requires both set-locality* and *a (near-)stateless reward.*

III. Threat model

In scope. A co-located tenant/process sharing C that issues arbitrary memory accesses and knows the Bouncer design (Kerckhoffs); its goal is to DoS the victim or steer C into a covert/side channel. Online-learning poisoning is an extension—the GATED state freezes learning. **Out of scope.** Physical/fault attacks and direct weight tampering (the base model assumes weights fixed at deploy). **Online-learning poisoning is explicitly out of scope for the base model:** an attacker who corrupts C's *weights* during TRUSTED operation

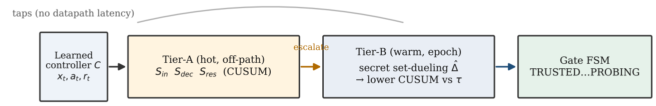


Fig. 1: Two-tier auditor. Cheap always-on Tier-A tripwires gate an expensive Tier-B competence confirmer; taps are read-only and nothing sits on the datapath critical path.

so that C looks competent on the secret leaders but harms the victim is not addressed here—freezing learning at GATED stops further poisoning but does not undo it, and PROBING re-trust would restore a poisoned policy. Defending learned *state* (vs. the steered-*input* threat we do treat) is future work. **The secret.** A per-epoch RNG seed assigning the dueling sets, never exposed through any architectural channel.

The adaptive adversary (headline). *Mimicry*: keep the Tier-A statistics in-distribution while degrading C. As we show in Proposition 1 and section XI, this is defeated by the secrecy and reseeding of the competence sampler—an attacker who can identify the dueling sets defeats it, which is precisely why the randomized-sampling ablation is the headline experiment.

Realizability—what we do and do not demonstrate. Bouncer's guarantees bound the cost of *any* in-domain competence degradation *however it arises*: benign distribution shift or adversarial steering are the same scalar event ($\Delta_W < \tau$) to the auditor. We realize the mimicry/steering adversary in the synthetic competence harness, where the latent stress u is the controllable knob; the real-ChampSim study (Section XI-J) drives the auditor with an *injected, controlled, labelled* competence degradation—a deliberately-bad eviction policy active on a known phase—*not* a demonstrated end-to-end real-silicon steering attack. Whether a co-located tenant can realizably steer a *specific deployed* controller to sub-resolution harm on real silicon, rather than in our harness, is *not established here and is future work*. The open question is the attacker's realizability; the defense's guarantee is degradation-source-agnostic, so the harness's injected degradation is a faithful stand-in for the auditor's input regardless of how a real degradation would be induced.

IV. Architecture

Bouncer is two-tier because cheap signals are spoofable and the unspoofable signal is expensive (Figure 1). The cheap tripwires run always; they gate the expensive confirmer.

Gate state machine. *TRUSTED*: C active everywhere, Tier-A on, Tier-B on a slow background duty cycle. Any Tier-A CUSUM firing → *SUSPECT*. *SUSPECT*: C still active, Tier-B duty cycle raised (more dueling sets, shorter window); the Tier-B $\hat{\Delta}$ CUSUM firing → *GATED*; Tier-A clearing for T_{clear} → TRUSTED. *GATED*: π_0 active everywhere *except*

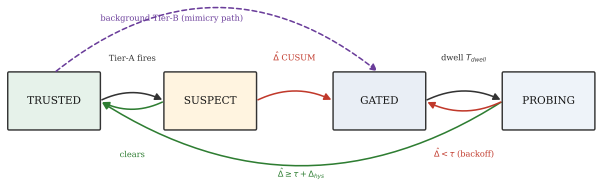


Fig. 2: The trust-gate FSM. Hysteresis and a minimum dwell prevent thrash; PROBING makes re-trust measured, not timed. The dashed path lets background Tier-B gate a pure-mimicry attack that keeps Tier-A silent.

the n_L Leader-L sets, which keep running C so $\hat{\Delta}$ stays measurable, online learning frozen, optional attribute/throttle; after a dwell T_{dwell} → *PROBING*. *PROBING*: C re-enabled on a small audited region only, Tier-B measures $\hat{\Delta}$ there; $\hat{\Delta} \geq \tau + \Delta_{hys}$ for $T_{reprobe}$ → TRUSTED, else → GATED with exponential backoff on T_{dwell}. Hysteresis (τ to enter, $\tau + \Delta_{hys}$ to leave) plus a minimum dwell prevent thrash; PROBING makes re-trust *measured*, not timed, defeating an attacker who waits out a fixed cooldown.

A design subtlety the build surfaced (Figure 2): a *pure* mimicry attack keeps Tier-A silent, so if SUSPECT were the only route to GATED the attack would evade the gate. We therefore allow the background Tier-B CUSUM to drive TRUSTED → GATED directly; Tier-A escalation only *accelerates* detection by raising Tier-B's duty cycle. The two tiers trade latency/energy, not correctness.

V. Tier-B: Competence via Randomized-Secret Set-Dueling

Mechanism. Partition the controller's resource (cache sets, memory banks, PC-hash buckets) into three pools, reseeded each epoch from a hardware RNG whose assignment is unobservable to software: *Leader-L* (n_L sets, always run by C), *Leader-F* (n_F sets, always run by π_0), and the *Follower* rest (run whichever leader currently wins—what the victim mostly experiences). This is exactly the set-dueling idiom already in silicon [14], [15]; Bouncer repurposes it from policy selection to policy trust. The estimator is

$$\hat{\Delta}_W = \bar{r}_L - \bar{r}_F, \tag{1}$$

the mean reward rate on the sampled L vs F sets over window W. Because Leader-L sets *ran* C and Leader-F sets *ran* π_0, $\hat{\Delta}$ is a *measured*, non-counterfactual quantity.

Concentration → detection latency (the key tradeoff). With m decisions per set per window and bounded rewards, $\mathrm{Var}(\bar{r}_{\text{pool}}) \leq r_{\max}^2/(4mn)$, so

$$\mathrm{Var}(\hat{\Delta}_W) \lesssim \frac{r_{\max}^2}{4m}\left(\frac{1}{n_L} + \frac{1}{n_F}\right) =: \sigma_\Delta^2. \tag{2}$$

To resolve a competence gap of size τ one needs $\sigma_\Delta \ll \tau$. Smaller pools or shorter windows give a noisier $\hat{\Delta}$, a higher CUSUM threshold H to hold the false-alarm rate, a longer detection delay D, and a larger safety- floor slack (Lemma 1). This chain—*pool size → estimator variance → CUSUM threshold → detection delay → regret floor*—is the central design dependency; every hyperparameter lives on it.

Detector. A one-sided lower CUSUM on $\hat{\Delta}_W$ catches a downward shift below τ:

$$C_t^- = \max\big(0,\, C_{t-1}^- + (K - \hat{\Delta}_W)\big), \quad K = \tau + \tfrac{\gamma}{2}, \tag{3}$$

alarming when $C_t^- > H$, and resetting on alarm (repeated-CUSUM, so it re-arms for the next change rather than latching after a long excursion). Standard quickest-detection theory [18], [19] motivates the CUSUM as the near-optimal detector for a downward shift under a false-alarm constraint (optimality is exact only for a known post-change law; here $\hat{\Delta}_W$ is an estimated, variance-inflated statistic, so we treat it as a well-motivated detector); we work the ARL/delay constants in Section VII.

VI. Tier-A: Cheap Tripwires

All Tier-A signals update on a sampled $1/k$ of decisions, a few adds/compares each, in $\leq$ low-KB SRAM, and *never gate on their own*—they only spend Tier-B budget, so their false positives cost a little energy, never correctness.

S_{in} **(input covariate shift).** A frozen reference sketch of D_{val} features: a random projection $P \in \mathbb{R}^{d \times p}$ ($p \approx 8$) feeding per-component quantile sketches; online, a windowed energy distance vs the reference. Cheap, high-sensitivity, and *spoofable*—the adversary can hold marginals fixed—which is exactly why it cannot be the trust signal.

S_{dec} **(decision-confidence collapse).** If C exposes value/Q/logits, monitor action-entropy collapse, max-Q margin, and (for online learners) TD-error magnitude. Pythia's reward and SHiP/perceptron confidence are already computed in silicon [20], [5], so S_{dec} is nearly free there.

S_{res} **(innovation residual).** A 16-weight linear forward model $g : (x_t, a_t) \to \hat{r}_t$ fit on D_{val}; online residual $e_t = r_t - \hat{r}_t$; two-sided CUSUM on e_t. This catches "the world responds differently than at validation" even when input marginals look clean—the architectural analogue of a Kalman innovation. It is the signal that separates a competence-aware monitor from an input-only one.

VII. Guarantees

A. Safety floor / bounded regret

Assumption 1. (A1) *per-decision reward* $\in [0, r_{\max}]$*;* (A2) *a* drop episode *is a window where the true competence $\Delta_t < \tau$ (the off-policy condition, consistent with the CUSUM reference $K = \tau + \gamma/2$); once one begins, Tier-B raises GATED within D windows w.p. $\geq 1-\delta$, where D is a high-probability bound (the mean delay $H/(K - \Delta_t)$ plus a tail term that shrinks with σ_Δ);* (A3) *the per-window false-alarm probability is $\leq \alpha = 1/\mathrm{ARL}_0$;* (A4) *a gate switch transient costs $\leq c_{sw}$;* (A5) *there are N_{ep} drop episodes over a horizon of T windows.*

Audit exposure (from the construction). The n_L Leader-L sets run C in *every* gate state (they must, so $\hat{\Delta}$ stays measurable and PROBING re-trust is *measured* rather than timed; Section V), and PROBING additionally re-enables C on a fraction ρ_{aud} of the followers. So the share of the resource still running C during a drop is $\phi_G = n_L/n_{sets}$ while GATED and $\phi_P = \phi_G + \rho_{aud}\Big(1 - \frac{n_L+n_F}{n_{sets}}\Big)$ while PROBING (ϕ_G=1.56%, ϕ_P=6.4% at the pinned operating point). Let T_{att} be the number of windows inside drop episodes.

Lemma 1 (Safety floor)**.** *Under Assumption 1, with probability* $\geq 1 - \delta N_{ep}$,

$$\sum_t \Big(r_t^{\pi_0} - r_t^{\text{Bouncer}}\Big) \;\leq\; \underbrace{N_{ep}\, D\, r_{\max}}_{\textit{detection}} + \underbrace{\phi_P\, T_{att}\, r_{\max}}_{\textit{audit exposure}} + \underbrace{\alpha\, T\, c_{sw}}_{\textit{false alarm}}. \tag{4}$$

Equivalently $R_{\text{Bouncer}} \geq R_{\pi_0} - [N_{ep}Dr_{\max} + \phi_P T_{att} r_{\max} + \alpha T c_{sw}]$*. The middle term is the price of continuous,* measured *auditability: it grows only at the dueling fraction* ϕ_P *of the attack duration, not at full resource. On any window where* C *is genuinely better and the gate is open* Bouncer $= C$*, so* $R_{\text{Bouncer}} \geq \max(R_{\pi_0}, R_{C\text{-trusted}}) - O(N_{ep}D + \phi_P T_{att} + \alpha T)$.

Proof sketch. Partition windows. (i) *GATED* windows run π_0 on the followers and Leader-F, but the n_L Leader-L sets keep running C (they are fixed, Section V); each contributes at most $\phi_G r_{\max}$ regret vs π_0. (i′) *PROBING* windows re-enable C on Leader-L plus the ρ_{aud} audited followers, contributing at most $\phi_P r_{\max}$ each. Over the T_{att} drop-episode windows these sum to at most $\phi_P T_{att} r_{\max}$ (bounding each by the larger ϕ_P). (ii) The $\leq D$ windows after each genuine drop, before the gate engages, run C everywhere and contribute $\leq r_{\max}$ each; there are N_{ep} of them, giving $N_{ep}Dr_{\max}$. (iii) False-alarm windows: at most αT, each costing $\leq c_{sw}$, giving $\alpha T c_{sw}$. (iv) Trusted windows where $C \geq \pi_0$ contribute non-positive regret. Summing the positive contributions (i, i′)+(ii)+(iii) yields eq. (4); the per-episode δ failure probability unions to δN_{ep}. The *tight* form charges the realized gap $(q_0 - \mu_C^{att})$ in place of $r_{\max}$ and the realized GATED/PROBING occupancy (ϕ_G per GATED, ϕ_P per PROBING window) in place of $\phi_P T_{att}$. ☐

The detection and false-alarm slack are the Section V knobs (the audit-exposure term is instead fixed by the design constant ϕ_P). The detection delay is $D \approx H/(K - \Delta_{true})$ and the false-alarm rate is $\alpha = 1/\text{ARL}_0$, with ARL_0 from Siegmund's corrected- boundary approximation [17]: for a one-sided CUSUM accumulating $(K - \hat{\Delta})$ with standardized drift $\delta = (K - \mu)/\sigma_\Delta$ and boundary $b = H/\sigma_\Delta + 1.166$,

$$\text{ARL}(\delta) \;\approx\; \frac{e^{-2\delta b} + 2\delta b - 1}{2\delta^2}. \tag{5}$$

In-control ($\mu \gg K$, $\delta < 0$), ARL_0 grows *exponentially* in H/σ_Δ, so false alarms are rare; out-of-control ($\delta > 0$), the delay is short. The guarantee thus degrades gracefully and *quantifiably* with detection delay and false-alarm rate.

B. Mimicry resistance

We state the property *conditionally*—it is exactly as strong as the secrecy and the within-domain exchangeability it assumes, and we are explicit that a *global* $\hat{\Delta}$ carries *no* guarantee against a redistributive attack.

Proposition 1 (Mimicry resistance, per-domain)**.** *Fix a contested domain* R *(e.g. a victim co-tenant's cache footprint) and run the dueling pools* within R*: draw Leader-L and the follower sets of* R *uniformly at random from* R *each epoch, unobservable to the adversary. Assume the within-*R *set outcomes are exchangeable up to a heterogeneity bias* $|b_R| \leq \beta$ *(Sec. below). If the adversary holds all aggregate Tier-A statistics at their validation values, then any input-only strategy that keeps* $\hat{\Delta}_R = \bar{r}_L - \bar{r}_F \geq \tau$ *forces* $\mathbb{E}[\textit{reward on the } R \textit{ followers}] \geq r^{\pi_0} + \tau - \beta$*: the victim cannot be degraded below the floor* while evading the gate. *Under a leaked fraction* f *of the within-*R *assignment, the adversary may spare* $f\, n_L$ *Leader-L sets, biasing* $\hat{\Delta}_R$ *upward by* $\leq f\,(\bar{r}^{clean} - \bar{r}^{att})$*; the* unprotected $(1-f)$ *of the degradation still enters* $\hat{\Delta}_R$*, so the gate fires once* $(1-f)(\bar{r}^{clean} - \bar{r}^{att}) \gtrsim H\,\sigma_{\Delta,R}$*, i.e. detection is intact while the leaked fraction stays modest (empirically up to* $f \approx 0.6$*, Section XI-E) and collapses sharply in the* 0.6–0.8 *band, well before* f=1 *(not a gradual roll-off toward* 1*).*

Proof sketch. Within R the Leader-L and follower sets are an exchangeable partition under the secret uniform draw, so their reward means satisfy $\mathbb{E}[\bar{r}_L] = \mathbb{E}[\bar{r}_{\text{foll}}] + b_R$ with $|b_R| \leq \beta$. An adversary blind to the partition cannot apply a set-dependent perturbation distinguishing them, so any input-only strategy raising $\bar{r}_L$ raises the follower mean to within β. Evasion ($\hat{\Delta}_R \geq \tau$) thus forces follower reward $\geq r^{\pi_0} + \tau - \beta$. Leaked knowledge of fraction f lets the adversary exempt $f\, n_L$ Leader-L sets; the remaining $(1-f)$ fraction carries the degradation into $\hat{\Delta}_R$, detectable once it clears $H\,\sigma_{\Delta,R}$. ☐

What this does and does not cover. (i) *Global* $\hat{\Delta}$ *is not enough.* A purely redistributive adversary that degrades the victim region while holding the *global* mean fixed evades a global $\hat{\Delta}$ outright; only per-domain dueling (the construction above, used in Section XI-E) detects it. (ii) *Heterogeneity bias.* Real cache sets / PC-buckets are not i.i.d. (hot/cold sets, conflict skew), so $\beta > 0$; the guarantee degrades by β, which secret *reseeding* each epoch shrinks toward the i.i.d. value by re-randomizing which physical sets are leaders. (iii) *Secrecy is load-bearing.* The whole argument rests on the adversary being blind to the within-R assignment—which is why the secrecy ablation (Section XI-E) is the headline experiment.

The coverage hole (an explicit slow-bleed attack on the budget). The proposition is genuinely scoped, not merely hedged: it covers a *declared* domain audited at finite resolution. From the concentration bound, the within-R gap resolvable with leader density ρ is $\delta_R = k\,\sigma_{\Delta,R} = k\, r_{\max}/\sqrt{2m\rho|R|}$, and—crucially—the leader count to audit at resolution δ_R is $n_{L,R} \geq k^2 r_{\max}^2/(2m\,\delta_R^2)$, *independent of* $|R|$. So under a fixed budget B there is a finest resolution $\delta_R^{\min}(B)$, and a redistributive adversary can pick a region small enough (or trickle harm slowly enough) that its per-window competence drop stays *below* $\delta_R^{\min}(B)$ forever—an undeclared, sub-resolution "coverage hole" that the safety-floor lemma, proven only over the *union of audited domains*, does not bound. This is the same hole the secrecy ablation walks on its f axis (TPR 1.0 $\to$ 0 as $f \to 1$). *We therefore*

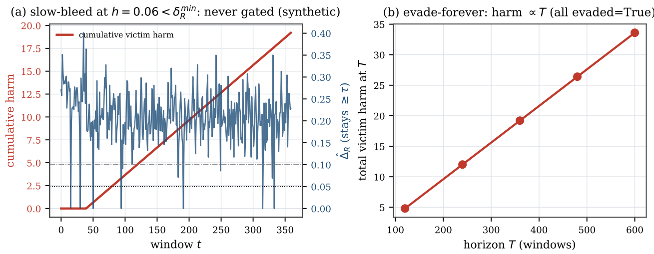


Fig. 3: The coverage hole, demonstrated (controlled harness, real Bouncer region dueling). (a) a slow-bleed at $h<\delta_R^{\min}$ accumulates unbounded victim harm while $\hat{\Delta}_R \geq \tau$ and no alarm fires; (b) it evades at every horizon, so total harm $\propto T$. The closed-form floor is $\delta_R^{\min}(B)=k\sigma_R \propto 1/\sqrt{B}$ $(0.19, 0.13, 0.094, 0.066$ for per-pool leader counts $n_{L,R}=2, 4, 8, 16)$: more leader budget shrinks the hole but never closes it.

claim mimicry resistance only for declared domains audited at resolution $\delta_R^{\min}(B)$ with an intact secret—not general, any-region security; the slow-bleed attack is a real, un-closed vulnerability that bounds the contribution honestly. Figure 3 demonstrates it against the deployed auditor: a constant per-window harm $h=0.06$ kept below $\delta_R^{\min}$ bleeds a victim region for the entire horizon—cumulative harm grows monotonically and unboundedly in T— while $\hat{\Delta}_R$ holds at $0.205 \geq \tau$ and the CUSUM never fires.

VIII. Attack-vs-drift triage

Both threats fail $\Delta_W < \tau$; their signatures differ enough to set the response prior (a secondary contribution—detection does not depend on it). Drift is slower and phase-marker-correlated and is *self-consistent* with a new-but- valid regime (the forward model S_{res} stays calibrated); an attack is abrupt, can pulse, is *targeted* at the decision boundary (high S_{res} with small S_{in}), and correlates with a specific co-tenant's schedule whose work yields it no benefit except moving the shared C. A small logistic model over the auditor-observable $(\Delta S_{in}, \Delta S_{res}, |\nabla\hat{\Delta}|,$ co-tenant- corr) suffices (Section XI P3 validates it under cross-validation); misclassification only mis-selects between equally-safe responses.

IX. Per-controller instantiation

Table I maps Bouncer onto four controller classes, ordered by *set-locality*. **Cache replacement** leads: its hit/miss reward is attributed to the very set whose eviction it decided, so the dueling $\hat{\Delta}$ is a clean, localized competence signal—the home turf. **Branch/perceptron** predictors are PC-indexed and likewise local. **Prefetching** is *not* set-local—a prefetch on set s rewards a different set s', so the faithful reward cannot be dualized (Section XI-J); it is the cautionary boundary. The **memory scheduler** is the hardest: no clean set-sampling on a shared command bus forces a *model-based* $\hat{\Delta}$, which weakens Proposition 1.

TABLE I: Per-controller instantiation, ordered by set-locality (whether reward and decision share a dueling set).

Controller	set-local?	r_t proxy	fallback π_0	dueling resource
Hawkeye/Glider (repl.)	**yes**	hit/miss	RRIP/LRU	cache sets
Perceptron branch	yes (PC)	correct	bimodal/gshare	PC slices
Pythia (RL prefetch)	**no**	acc.$\times$timel.	next-line/off	PC buckets
RL mem. scheduler	partial	$-$lat./+BW	FR-FCFS	banks/chan.

X. Methodology

Bouncer's guarantees rest only on bounded reward and a partitionable resource (assumptions A1–A5, Equation (2), Lemma 1 and proposition 1), which we argue makes them *environment-agnostic*. We therefore validate the *mechanism* in *one* faithful, controllable synthetic harness—instantiated three ways in Section XI(P3)—that models a learned controller, a safe fallback, benign drift, and three adversaries, and *also* build the same auditor as a real ChampSim L1D prefetcher module run on SPEC CPU2017 (Section XI-J). *The controlled quantitative claims (latency, floor, mimicry survival, secrecy) come from the harness; the ChampSim integration reports real IPC on a 3-trace SPEC suite as a proof of deployment, not a full sweep.* The harness models the prefetcher setting as the anchor: a resource of $n_{sets}=2048$ sets; a latent per-decision *stress* $u \in [0, 1]$ (how far the input is pushed toward C's boundary) through which both drift and steering act and which the auditor never observes; controller reward $\mu_C(u) = r_{\max}\,\mathrm{clip}(0.8 - 0.7u)$ collapsing under stress and a flat floor $\mu_0 = 0.5 r_{\max}$; bounded rewards drawn so Var $\leq r_{\max}^2/4m$ exactly. The IPC transfer IPC $= 0.6 + 1.4\bar{r}$ lets us report the systems metrics. *These two maps are not load-bearing for the qualitative guarantees:* Section XI-G re-runs the floor and both headline P4 results across a family—two collapse shapes (clip, logistic) at two slopes, an alternative IPC map, a shifted fallback floor $q_0 \in [0.4, 0.6]$, and a joint $\mu_C\times$IPC change—and finds the qualitative guarantees invariant; only the exact percentages move. Detection *power* is the one quantity we do not claim is invariant: the σ_Δ bound predicts it must degrade once $|\Delta_W|$ approaches σ_Δ, but our stress sweep stops short of that regime—TPR stays 1.0 down to the smallest gap tested ($|\Delta_W|=0.015 \gg \sigma_\Delta$), so the predicted roll-off is not yet exercised. We pin the Section V operating point at $n_L=n_F=32$, $m=64$ ($|W|\approx 1.3\times 10^5$ decisions), $\tau=0.05$, $K=0.10$, $H=0.8$. Conditions are clean, benign-drift, and three attacks (broad DoS, adaptive mimicry, regional covert); baselines are no-auditor, input-OOD-only (S_{in}), oracle-Δ, and always-fallback. The harness and the ChampSim skeleton are released.

Hypothesis discharge. Table II maps every premise of Lemma 1 (A1–A5) and Proposition 1 to where it is established, measured, or assumed—and, crucially, records the *one* premise (A1's stateless sub-condition) that the real-systems study shows *fails* for a real cache, which is the substance of the set-locality refinement (Remark 1).

TABLE II: Hypothesis discharge: where each premise is established (E), measured (M), or assumed (A). The stateless half of A1 is the load-bearing one—it holds by construction in the harness and *fails, measured,* on a real cache.

Premise	Discharged
A1 bounded reward $\in [0, r_{\max}]$	E/M: synthetic by construction; real hit/miss $\in \{0, 1\}$.
A1 *stateless* reward (i.i.d. across epochs)	A (synthetic, by construction); **fails, measured, on a real cache** (Remark 1, Section XI-J: reseeded $\hat{\Delta}\approx 0$ vs fixed-leader 0.23).
A2 detect within D w.p. $\geq 1-\delta$	E: $\sigma_\Delta \to$CUSUM delay chain (Section V); M: P0/P1 (1-window).
A3 false alarm $\leq 1/\mathrm{ARL}_0$	E: Siegmund ARL (Section V); M: P2 ROC, sensitivity sweep.
A4 switch transient $\leq c_{sw}$	A (analytical); M: P1 one-window dip $\leq Dr_{\max}$.
A5 N_{ep} episodes over T	A (definitional, counts episodes).
Prop. 1 finite resolution $\delta_R^{\min}(B)$	E: Equation (2); the sub-resolution slow-bleed is the named open hole (M: Figure 3).
Prop. 1 intact secret (leak f=0)	E/M: secrecy ablation (P4), TPR collapses as $f\to 1$.

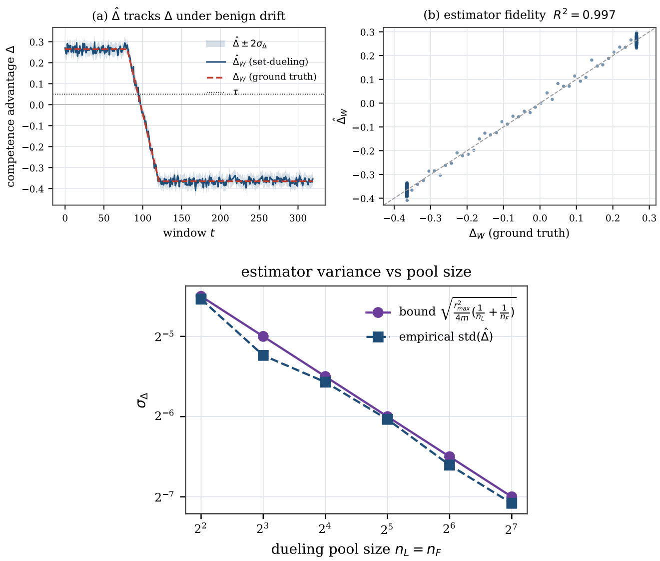


Fig. 4: P0. (top) $\hat{\Delta}$ tracks ground-truth Δ (R^2=0.997); (bottom) empirical $\mathrm{std}(\hat{\Delta})$ matches the bound eq. (2).

XI. Evaluation

A. P0: the estimator tracks ground truth

Figure 4(a) shows $\hat{\Delta}_W$ tracking the ground-truth Δ_W through a benign drift that carries competence from positive through zero to negative; the fit is R^2=0.997 with RMSE $0.0141 \approx \sigma_\Delta$=0.0156. This is a *controlled-harness* fidelity at the synthetic operating point (m=64); on a real trace with far fewer samples per bucket the estimator is correspondingly noisier (Section XI-J). Figure 4(b) confirms the concentration bound eq. (2): across a pool-size sweep the empirical std of $\hat{\Delta}$ sits at 0.85–0.98× the bound, a tight upper envelope (the bound uses $\frac{1}{4} \geq p(1-p)$).

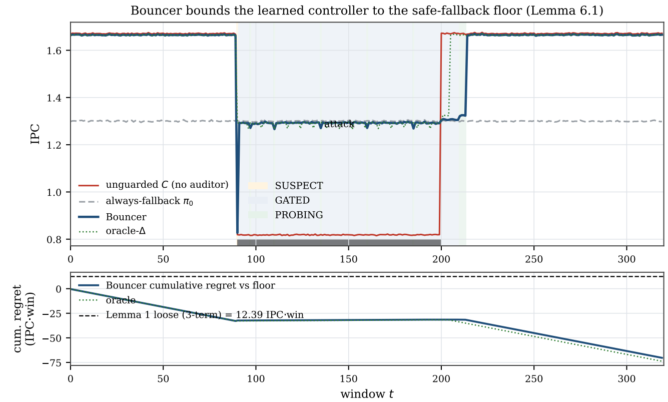


Fig. 5: P1. Bouncer bounds the controller to the fallback floor under attack and re-trusts after it ends; the unguarded controller crashes below the floor.

B. P1: the safety floor holds

Figure 5 is the constructive hedge made real. Under a poisoning attack (on at window 90, off at 200) the *unguarded* controller crashes far below the fallback floor; Bouncer detects in **one window** here (the worked mean delay is $D \approx 1.8$ windows; one window is the best case at this high SNR), floors to π_0, and—crucially—*re-trusts* 14 windows after the attack ends via PROBING, not a fixed timer. The *steady-state* floor violation is 0.63%, and Lemma 1 now *predicts* it: the audit-exposure term is exactly the Leader-L sets that keep running C under attack, so the floor sits at $\phi_G(q_0 - \mu_C^{att})\,\mathrm{slope}/\mathrm{IPC}_{\pi_0} =$ 0.58%, rising to 0.65% once the GATED/PROBING duty is blended (a 1.76% effective exposure), bracketing the measured 0.639%. The only larger excursion is the lone pre-detection window, which dips 36% below the floor: exactly the bounded $\leq Dr_{\max}$ term of Lemma 1, and the price of a one-window detection delay. Performance recovered is 0.55 (Bouncer recovers to the *fallback* floor, not to clean, since the attack genuinely removed C's value), the clean tax is 0.9965 (above the 0.99 target; the tax is exactly the n_F Leader-F sets that permanently run π_0), and the bottom panel shows cumulative regret going *negative* (Bouncer beats the floor by capturing C's upside when trusted), the measured worst-case positive excursion staying under the corrected three-term loose bound of Lemma 1 (12.4 IPC·win). Parking the Leader-L sets on π_0 during GATED would zero the exposure term but blind the *measured* re-trust PROBING relies on; we keep the auditable design and pay the predicted 0.63%.

C. P2: ROC, the §V chain, and overhead

Figure 6 shows the competence detector achieving TPR=1.0 at FPR≤0.05 on *both* broad and mimicry attacks, while the input-OOD detector S_{in} works on the broad attack but collapses to TPR=0 under mimicry. Figure 7(a) traces the Section V chain: as the pool shrinks from 128 to 2, σ_Δ rises $0.016 \to 0.125$, the calibrated H rises $0.05 \to 0.35$, and detection latency rises $0 \to 3.4$ windows, tracking the analytic

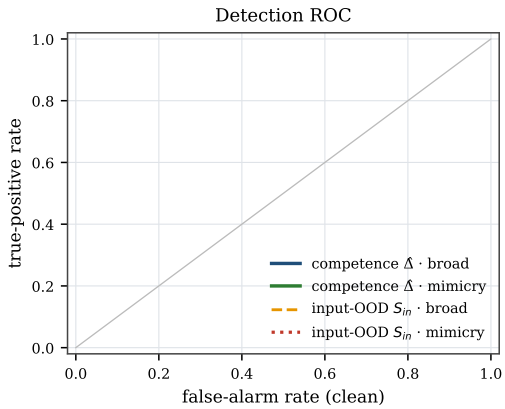
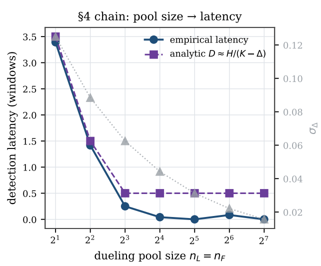
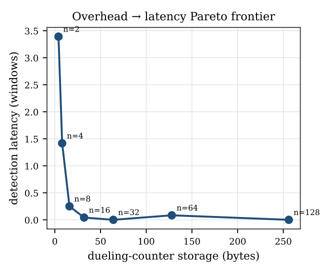


Fig. 6: P2 ROC. The competence detector dominates; S_{in} is blind to mimicry.



(a)



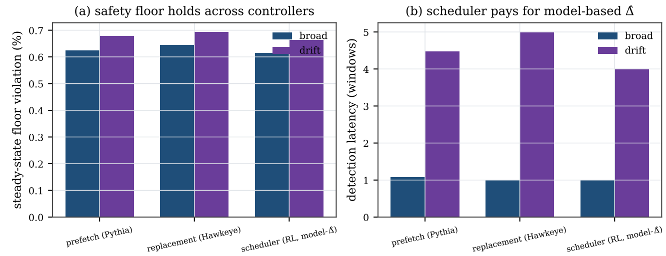
(b)

Fig. 7: P2. (a) pool size $\rightarrow \sigma_\Delta \rightarrow H \rightarrow$ latency, with a knee near n=8; (b) overhead/latency Pareto frontier.

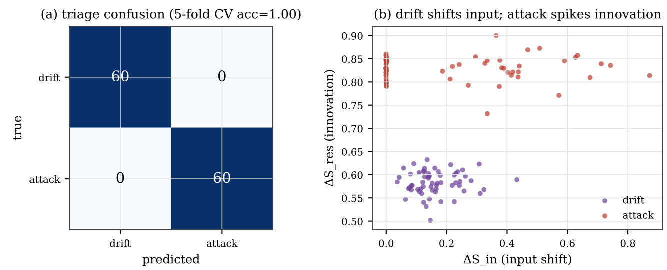
$D \approx H/(K - \Delta)$. There is a clear knee near $n \approx 8$: above it detection is gap-limited and pool-insensitive, so larger pools only add overhead; Figure 7(b) is the resulting overhead/latency Pareto. The full Section XII budget is $\sim$336 bytes, $\sim$0.13% of a 256 KB SRAM, and adds no cache-access-critical-path latency by construction (analytical; RTL timing future work).

D. P3: generality and triage

The same auditor wraps three controller classes (Figure 8(a)): a prefetcher and a replacement policy with secret set-dueling, and a memory scheduler with a model-based $\hat{\Delta}$. The steady-state floor holds for all three ($\leq 0.7\%$ violation) under both broad attack and drift; the scheduler pays for its weaker estimator with longer latency and lost mimicry resistance. The attack-vs-drift triage (Figure 8(b)) reaches 100% *5-fold cross-validation* accuracy over 120 episodes on *auditor-observable* features (the abruptness is the slope of the estimated $\hat{\Delta}$, not of ground truth); a drop-one-feature ablation shows the result is robust to removing *any* single feature—including the modeled co-tenant-correlation signal—so the separability comes from the genuine signature gap (gradual input-shift drift vs. abrupt boundary-seeking attack), not from one label-correlated input.



(a)



(b)

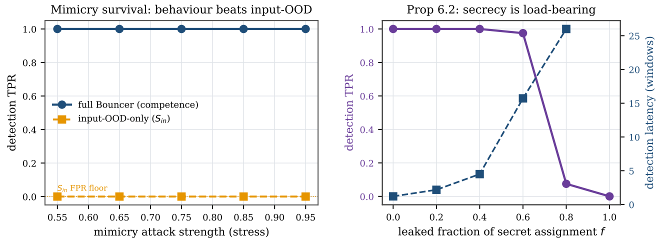
Fig. 8: P3. (a) the floor holds across controller classes; (b) attack-vs-drift triage (acc 1.0).

E. P4: mimicry survival and the secrecy ablation

This is the result the paper is built around (Figure 9). **(a)** Under an adaptive mimicry adversary that holds S_{in} in-distribution while degrading C, the input-OOD-only baseline detects *none* of the attacks (0/40 episodes at every strength), while full Bouncer detects *all* (40/40; clean false alarms 0/40 for both). Over 50 independent seeds (Figure 10(a)) this is full-Bouncer TPR 1.0 (Wilson 95% $[0.93, 1.0]$) vs. input-OOD 0.0 ($[0, 0.07]$), with clean FPR 0.0 ($[0, 0.07]$) for both; detection latency is 1.0 window $[1.0, 1.0]$, the steady floor violation 0.639% $[0.637, 0.641]$, and the clean tax 0.99653—all with tight 95% intervals. Measuring *realized competence* rather than input distribution is decisive. **(b)** The secrecy ablation realizes Proposition 1 empirically: against the strongest, redistributive covert adversary, with the dueling pools drawn *within* the victim region (an effective in-region pool of $\sim$4 Leader-L sets, hence a few windows of latency), detection power degrades smoothly as the secret leaks— TPR=1.0 for leaked fraction $f \leq 0.4$, 0.975 at f=0.6, collapsing to 0.075 at f=0.8 and 0 at f=1.0, with latency rising $1.2 \rightarrow \infty$ windows. The robustness in (a) is purchased *entirely* by the secrecy of the dueling sets.

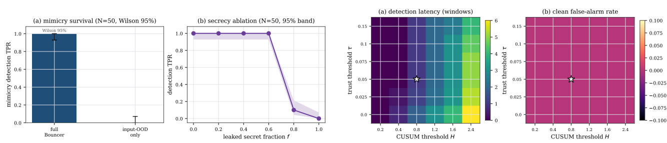
F. Robustness: sensitivity and confidence

The headline numbers are not single-seed artifacts (Figure 10(a), N=50, Wilson 95% intervals above), nor a knife-edge operating point: sweeping the trust threshold τ and CUSUM threshold H over a 6×6 grid (Figure 10(b)), **28/36** cells achieve $\leq$ 3-window detection with clean FPR $\leq$ 5% and no missed episodes—a broad robust plateau around the chosen (τ=0.05, H=0.8). Detection degrades gracefully and predictably toward the plateau edges (small H raises false



(a) (b)

Fig. 9: P4 (headline). (a) full Bouncer holds TPR=1 under mimicry where input-OOD collapses to 0; (b) detection power vs. the leaked fraction of the secret assignment (Proposition 1).



(a) (b)

Fig. 10: Robustness. (a) headline results over 50 seeds with Wilson 95% intervals; (b) sensitivity of detection latency and clean false-alarm rate to (τ, H)—the operating point ($\star$) sits on a broad plateau.

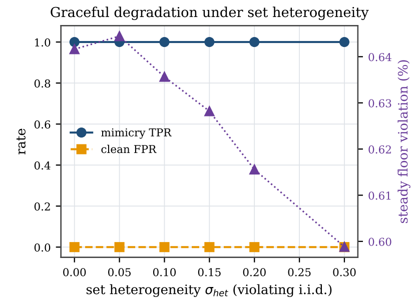


Fig. 11: Graceful degradation when the i.i.d.-set assumption is violated: under set heterogeneity σ_{het} up to 0.30, mimicry TPR stays 1.0 and the floor holds; only detection latency drifts up ($1.0 \to 1.7$ windows).

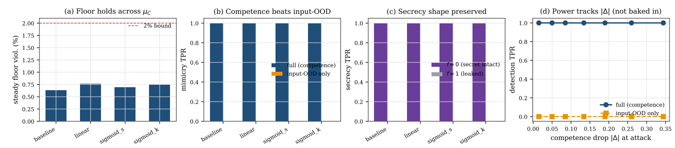


Fig. 12: Transfer-function sensitivity. Across collapse-curve shapes (a), the competence-beats-input mimicry result (b) and the secrecy-ablation shape (c) are invariant; (d) across the stress sweep the competence detector holds TPR=1.0 (vs. input-OOD 0) for every gap down to the smallest tested ($|\Delta_W|$=0.015 $\gg \sigma_\Delta$); the sweep does not enter the $|\Delta_W| \sim \sigma_\Delta$ regime where power must roll off.

alarms; large τ raises misses), exactly as the Section V chain predicts.

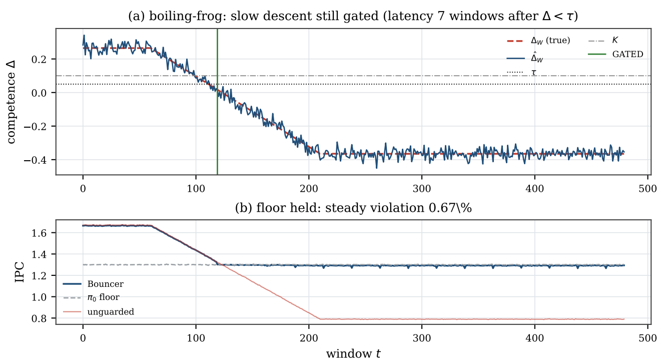
Misspecification robustness. Because the harness reward is built from the theorems' assumptions (bounded reward, near-i.i.d. sets), we test whether the results survive violating them. We give each set a fixed competence offset $\sim \mathcal{N}(0, \sigma_{het})$ (hot/cold sets, conflict skew), so Leader-C and Leader-F pools are exchangeable only in expectation—the β-bias regime of Proposition 1. Sweeping σ_{het} from 0 to 0.30 (Figure 11), mimicry detection stays at TPR 1.0 and clean FPR at 0, the steady floor holds (0.60–0.64%), and detection latency degrades only gently ($1.0 \to 1.7$ windows)—because secret *reseeding* re-randomizes which physical sets are leaders each epoch, averaging the heterogeneity bias down. The guarantees are thus not an artifact of the i.i.d. reward model.

G. Transfer-function sensitivity

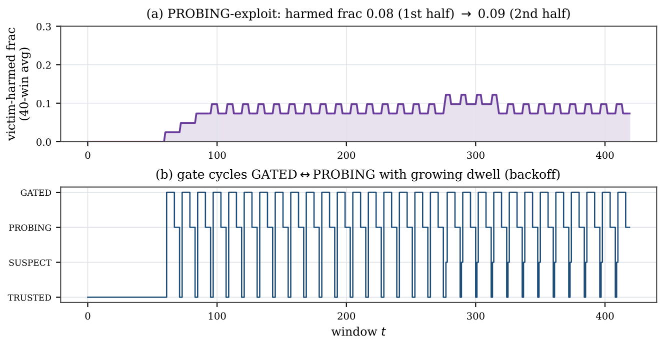
A reviewer's natural objection is that the headline percentages are artifacts of the chosen controller-collapse curve $\mu_C(u)$ and IPC map. We re-run the floor (P1) and both P4 headlines across a family (Figure 12): the collapse curve as clip and as a logistic at two slopes; the IPC map steeper and concave; a *joint* $\mu_C \times$IPC change; and—the load-bearing axis—the fallback floor q_0 swept over $[0.4, 0.6]$. **The qualitative guarantees are invariant**: across every cell the steady floor violation stays $\leq 0.9\%$ with a genuine drop detected, mimicry TPR is 1.0 for competence vs 0 for the input-OOD monitor, and the secrecy ablation keeps its TPR-vs-f shape; only the exact percentages move (clean tax 0.49–0.52% across maps, on a reduced 20-episode budget per cell). Detection *power* is the one quantity we do not claim invariant: a sweep of the attack stress toward the competence crossing holds TPR at 1.0 for every gap down to the smallest tested ($|\Delta_W|$=0.015, still $\gg \sigma_\Delta$); this reduced-episode sweep does not reach the $|\Delta_W| \sim \sigma_\Delta$ regime where the σ_Δ bound predicts roll-off, so within the tested range the detector's firing reflects a real competence signal above the resolution floor, and a sweep into the $|\Delta_W| \sim \sigma_\Delta$ regime (where power must fall) is left to future work. *Scope:* this establishes robustness to the *shape* of the collapse curve, the IPC map, and the floor location; the Tier-A feature/confidence maps are not swept (the headline competence-beats-input result is a Tier-B property).

H. Adaptive timing adversaries

Two threat-model-completeness attacks (Figure 13). **Boiling-frog** (Figure 13(a)): a very slow, stealthy degradation tuned to keep the per-window change tiny. It cannot evade—a one-sided CUSUM integrates the *level* $(K - \hat{\Delta})$, not the slope, so once $\hat{\Delta} < K$ it accumulates regardless of how slowly it arrived. Bouncer gates 7 windows after the true $\Delta < \tau$ crossing (vs. 1 for an abrupt attack); the slow descent only *delays* detection, and the steady floor still holds (0.67%). **PROBING-exploit** (Figure 13(b)): a closed-loop attacker that watches the gate and backs off during GATED/PROBING to pass the audit, resuming when TRUSTED—the "wait out the cooldown" strategy. Because re-trust is *measured* (PROBING



(a)



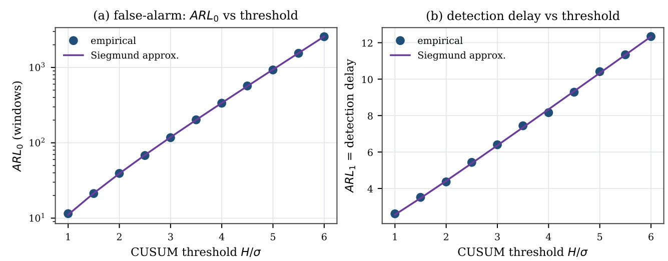
(b)

Fig. 13: Adaptive timing adversaries. (a) boiling-frog is still gated (a CUSUM integrates level, not slope); (b) the PROBING-exploit attacker is bounded to 8.6% harmed windows by measured re-trust + dwell backoff.

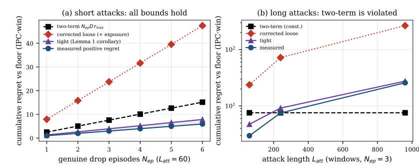
audits $\hat{\Delta}$) and the dwell backs off exponentially, the victim is harmed in only 8.6% of windows and the attacker spends $\sim 11\times$ more windows idle than effective; there is no fixed cooldown to wait out.

I. Theory validation

Figure 14(a) checks the worked CUSUM constants on an i.i.d. Gaussian signal across a threshold sweep: the empirical ARL_0 and detection delay match Siegmund's approximation eq. (5) (geometric-mean ratio 0.998, e.g. ARL_0 empirical 927 vs. theory 938 at $H{=}5\sigma$; a small H-dependent bias remains). This sweep is a *formula sanity check* covering the small-drift, heavy-overshoot regime where the 1.166 correction matters; the deployed detector runs in the opposite, strong-drift regime ($(K-\Delta)/\sigma_\Delta \approx 28$ at $n_L{=}32$), where the delay collapses to the deterministic limit $D \approx H/(K-\Delta)$ and ARL_0 is astronomically large (so $\alpha \approx 0$). Figure 14(b) has two panels. Panel (a) sweeps N_{ep} at short attacks ($L_{att}{=}60$): the old two-term $N_{ep}Dr_{\max}$, the corrected three-term loose, and the *tight* form (realized gap $q_0-\mu_C^{att}$ with ϕ_G/ϕ_P occupancy) all envelope the measured worst-case positive regret ($\alpha T c_{sw} \approx 0$ here); at $N_{ep}{=}6$ the two-term reads 15.1, tight 7.8, measured 5.9 IPC·win, the tight bound hugging the data. Panel (b) sweeps the attack length L_{att}: because the audit-exposure



(a)



(b)

Fig. 14: Theory. (a) ARL matches Siegmund; (b) the safety-floor bound envelopes measured regret.

regret grows linearly in L_{att} while the two-term $N_{ep}Dr_{\max}$ is constant, the two-term is *violated* for long attacks (measured 25.8 vs two-term 7.6 at $L_{att}{=}960$), whereas the corrected three-term of Lemma 1 continues to envelope. The tight form is thus a corollary of the corrected lemma, not a separate accounting.

J. Real-systems integration: the set-locality boundary

The auditor compiles and runs in real ChampSim as *both* a cache-replacement module (set-local by Definition 3—structurally the clean case, but stateful; see below) and an L1D prefetcher module (*not* set-local). The two together test the set-locality characterization on real hardware traces: the prefetcher case exhibits the de-localization boundary in detail, and the replacement case shows the clean side is set-local yet stateful—a second boundary (the secret reseed confounds a path-dependent reward), rather than a real-hardware clean-case validation. We present the prefetcher case first because it is where the boundary *bites*—it is the cautionary result, not the headline.

Prefetcher (the cautionary, non-set-local case). We built a real **ChampSim L1D prefetcher module**—the same set-dueling estimator, CUSUM, and gate FSM C++ as Section XII—with the "learned controller" a per-IP stride prefetcher, the fallback prefetch-*off*, the reward the per-set cache-hit rate, and the attack a cache-polluting corruption (the dueling "sets" here are virtual block-address buckets, $\mathrm{blk} \bmod 2048$, not physical L1D sets). It compiles and runs on SPEC CPU2017 with **no added work on the cache-access critical path by construction** (the confirmer/CUSUM/reseed/FSM are off-path and epoch-grained; the per-access cost is only a 2-bit pool-tag lookup and output mux—RTL timing is future work, Section XII): the confirmer/CUSUM/reseed/FSM are off-path and epoch-grained, and the per-access work is a 2-bit pool-tag lookup and an

TABLE III: ChampSim L1D suite (9M-insn SPEC). The floor cap bounds the attack on every trace; the cache-hit reward proxy sets the clean tax.

SPEC trace	prefetch benefit	attack loss (unguard)	Bouncer clean tax
600.perlbench (compute)	$+1.7\%$	0.3%	1.7%
619.lbm (streaming)	≈ 0 (hit-rate)	7.9%	2.3%
654.roms (mem-bound)	$+12.7\%$	5.2%	11.2%

output mux—the same kind of table read DIP/DRRIP already carry. Because the real $\hat{\Delta}$ is far noisier than the synthetic one (only $m \approx 19$ accesses/bucket/window land, so $\hat{\Delta}$ swings in $[-0.8, +0.7]$), we ran at a looser operating point (τ=0, H=0.25) than the synthetic (τ=0.05, H=0.8, m=64); we disclose this rather than claim the synthetic R^2=0.997 estimator fidelity transfers (it does not—that is a controlled-harness result).

Across a three-trace suite (Table III, Figure 16) the candid lesson is: the auditor is only as good as its competence reward. The *floor cap is reliable*—on every trace Bouncer ends at the prefetch-off floor, so the corruption attack's worst case is bounded: on streaming 619.lbm a corrupted prefetcher costs the unguarded config 7.9% IPC while Bouncer (already at the floor) is unharmed; on compute-bound 600.perlbench the attack is nearly harmless (0.3%) and so is the cap. But the per-set cache-hit reward *proxy* mis-estimates prefetch competence, so the clean tax ranges 1.7–11%: on memory-bound 654.roms stride prefetching genuinely helps IPC (1.14 vs the 1.01 off floor), yet the hit-rate signal misses that benefit (it comes from latency/MLP, not raw L1D hit count), so Bouncer over-gates. Two further honesty points: the gate is *competence*-driven, not attack-label-driven—on lbm it engaged before the attack onset, so this is a floor cap, *not* attack-specific detection; and the TRUSTED→GATED transition *dynamics* are shown where competence is controllable (the synthetic Figure 5), since no available naive-prefetcher reward exhibits a clean pre-attack TRUSTED state here.

It is tempting to blame the hit-rate proxy and propose the controller's *own* reward as the fix—but we tested exactly that and it is *insufficient*. Holding $(\tau, H, \text{window}, \text{partition}, \text{FSM})$ fixed and swapping *only* the reward, from per-set cache-hit to the prefetcher's own accuracy×timeliness, moves the roms clean tax only $11.2\% \rightarrow 9.7\%$ and still loses 5% to the attacked-but-ungated prefetcher. The reason is a **fundamental tension in what set-dueling can measure**: the cache-hit reward is *set-local* (attributable to the dueling set) but a poor utility proxy, whereas the own-accuracy reward is a faithful proxy but *not set-local*—a prefetch triggered on a Leader-L set targets a *different* address that lands on a *different* set, so the dueling mis-credits it and $\hat{\Delta}$ collapses to ≈ 0. The dominant lever is therefore the observation window / signal SNR, not reward fidelity: lengthening the epoch (window $40\text{k} \rightarrow 200\text{k}$) removes the over-gating even with the crude proxy. This is a genuine limitation of set-dueling *for prefetchers*, whose benefit is inherently de-localized; *cache replacement*—where the decision and its hit/miss outcome share a set—is set-local by construction (the paper's "home-turf" condition): the structurally clean case in theory, though Section XI-J shows a real cache's stateful reward adds a second, stateless-reward requirement before that cleanness is measurable. The milestone this real-simulator boundary study sets up is accordingly sharper: a long-epoch, *set-local* competence signal (latency-weighted, replacement-style), not merely "the controller's own reward."

Replacement (set-local but *stateful*—a sharper boundary). The same auditor C++ compiles and runs as a real ChampSim *LLC replacement* module: DRRIP-style dueling on *physical* cache sets (learned C = an aggressive LIP-style insertion, fallback π_0 = SRRIP-HP) scored by the demand-hit reward. Three real findings (Figure 15, 623.xalancbmk), each of which *sharpens* rather than confirms the easy "home-turf" story. *(i) The prior "$\hat{\Delta} \approx 0$" was a software artifact, not sampling.* A ChampSim replacement module's per-access hook fires only if the module also provides a fill hook (`has_cache_fill`); without it the module sees *hits only*, the demand-hit reward collapses to 1, and $\hat{\Delta}$ is *identically* 0. Adding the fill hook makes $\hat{\Delta}$ a real signal—so the earlier null was a missing callback, not a horizon limit. *(ii) The whole-cache competence gap is real and large:* forcing every set to the learned policy gives a 33.6% LLC hit rate vs 4.8% for the fallback. *(iii) Yet the secret per-epoch reseed—the very mechanism that buys mimicry resistance—**confounds** the per-window estimate for this controller.* With reseeding, $\bar{r}_L \approx \bar{r}_F$ and $\hat{\Delta}$ sits at noise (clean mean 0.005) despite that 29-point whole-cache gap; with *fixed* leaders (reseed off, DRRIP's own configuration) the *same* estimator resolves it (clean mean $\hat{\Delta} = 0.23$). The reason is that a replacement policy's reward is *path-dependent*: a set must run one policy *consistently* to accumulate the policy-specific cache state DRRIP's PSEL counter integrates, and reseeding each epoch erases it. **Set-locality is thus necessary but not sufficient: a secret, reseeded competence audit additionally requires a (near-)*stateless* reward**—which the synthetic harness's i.i.d. per-decision reward (A1) satisfies and a real cache does not at per-window grain. This is a genuine secrecy/measurability tension, not a tuning issue: the clean-case competence and transition dynamics consequently live in the synthetic harness (a faithful *stateless* set-local model, Figure 5), and the deployment milestone is a reseed schedule slow enough to amortize cache-state warmup—a trade-off this result makes precise.

XII. Overhead Budget

Table IV is the *analytical* overhead budget (an RTL-level measurement is future work). Set-dueling counters reuse the existing ATD/sampler; the Tier-A sketches and the 16-weight forward model sit in adjacent SRAM and update on a sampled $1/k$ of decisions; the CUSUM detectors are two registers and a comparator each. Total ~ 336 bytes, $\sim 0.13\%$ of a 256 KB SRAM, an estimated $<1\%$ energy, and—because the confirmer is sampled, off-path, and epoch-grained—**no added latency on the cache-access critical path by construction** (analytically; RTL timing is future work), the non-negotiable

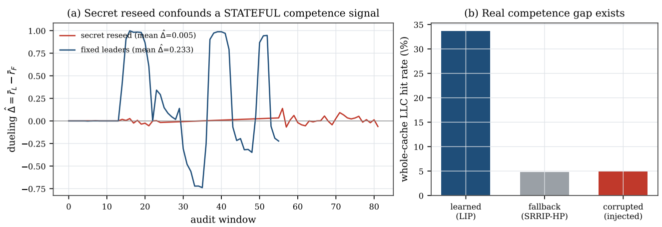


Fig. 15: Real ChampSim *replacement* (623.xalancbmk). (a) The secret per-epoch reseed—load-bearing for mimicry resistance—*confounds* the per-window competence estimate for a *stateful* reward: with reseeding $\hat{\Delta}$ sits at noise; with fixed leaders the same estimator resolves the gap. (b) The whole-cache gap is real (learned vs fallback); the `cache_fill` fix makes $\hat{\Delta}$ nonzero (it was identically 0). Set-locality is necessary but not sufficient—a secret, reseeded audit also needs a (near-)stateless reward. This is a boundary result; the clean-case dynamics remain in the synthetic harness.

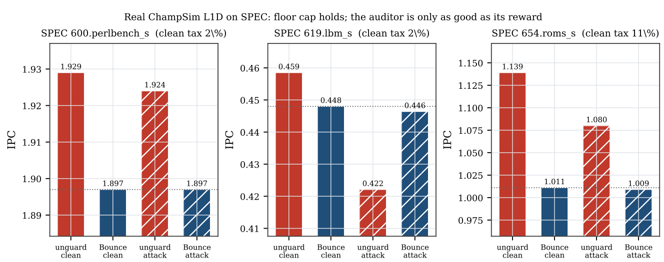


Fig. 16: Bouncer as a real ChampSim L1D prefetcher across a SPEC suite. On every trace Bouncer ends at the prefetch-off floor (the cap), bounding the corruption attack's worst case (lbm: 7.9% unguarded loss cannot reach the gated victim). The per-set cache-hit reward proxy sets the clean tax (1.7–11%): roms shows the failure mode—a genuinely-helpful prefetcher under-credited and over-gated. Swapping to the controller's own reward does *not* fix this (only $11.2\% \rightarrow 9.7\%$); a prefetcher's benefit is not set-local, so set-dueling needs a longer epoch or a set-local signal (replacement is the *set-local* case—though a real cache's stateful reward is a further boundary; see Section XI-J).

constraint. The only per-access work is the 2-bit pool-tag lookup and output mux (a table read comparable to existing DIP/DRRIP set-dueling), not the estimator or gate logic.

XIII. Related work

Learned microarchitectural controllers. A growing body of work replaces hand-tuned heuristics with online or offline-trained predictors across every major structure Bouncer targets. RL memory schedulers [4] and RL prefetchers [1] learn performance-coupled policies online; learning-based cache replacement imitates Belady's optimum with PC-indexed predictors [2], distilled neural models [3], or signature-based hit predictors [20]; and neural and geometric-history branch predictors [5], [6] have become the dominant adaptive prediction

TABLE IV: Overhead budget (analytical).

Component	Storage	Per-decision work
Set-dueling counters	64 B (reuse ATD)	reuse existing
S_{in} RP + quantile sketch	224 B	$\leq$ few adds on $1/k$
S_{res} forward model g	32 B	16 MACs on $1/k$
CUSUM detectors ($\times 4$)	16 B	2 add/cmp each
Total	**∼336 B**	off critical path

substrate. These designs share a failure surface: their advantage over a simple baseline (LRU/RRIP, a stride prefetcher, a bimodal predictor) is workload- and phase-dependent and can silently invert under distribution shift. Bouncer is agnostic to which of these controllers it wraps; rather than proposing a new policy, it provides the missing runtime guarantee that any of them never costs more than its own known-safe fallback.

Set dueling and adaptive policy selection. Set dueling [14] dedicates a handful of sampled sets to each of two competing policies and lets a single saturating counter steer the followers, a mechanism extended to re-reference prediction [15], shared-cache thread-aware insertion [21], and evicted-address filtering [22], and rooted in the sampling-monitor methodology of utility-based partitioning [23] and sampling dead-block prediction [24]; a recent retrospective documents its industrial adoption and limits [25]. Bouncer borrows the dedicated-sets-plus-counter substrate but repurposes it from policy *selection* to policy *trust*: where DRRIP duels two heuristics to pick the empirical winner, Bouncer duels the learned controller against its fallback to *certify* a competence advantage and, crucially, keeps the run-C/run-fallback assignment *secret*. Classic set dueling assumes cooperative policies and is content to follow whichever wins; Bouncer must instead defend against a controller that may be steered or drifting, which is why its dueling sets feed a change detector and a trust gate rather than a preference counter.

Safe RL, runtime assurance, and Simplex. The closest architectural ancestor is Simplex [26]: a simple, formally verified safety controller bounds an unverifiable high-performance one via a decision module. Shielding [27] synthesizes a correct-by-construction shield from a temporal-logic specification, and constrained- and Lyapunov-based safe RL [28], [29], [30], [31] keep policies inside a certified-safe set, with safe policy improvement [32], [33] reverting to the data-collecting baseline wherever improvement is not statistically supported (surveyed in [34]). Bouncer inherits the ”use simplicity to control complexity” posture and the bootstrap-to-baseline reflex, but departs in what it must prove. Simplex and shielding require a *verified model of the safety envelope*; constrained RL requires a known cost function. Bouncer needs neither: microarchitectural ”safety” is not a state-space invariant but a relative performance floor, so it certifies only that the learned controller's *realized reward* beats the fallback's on randomized dueling sets, with a bounded-regret safety floor (our Lemma) that holds without any reachability proof or verified plant model. The closest relaxation of Simplex's model requirement, Black-Box Simplex [35], still forwardsimulates a reachability/admissibility check and defines safety as a state-space invariant; Bouncer instead defines it as a measured relative-competence floor and needs no plant model at all. Concurrent runtime-assurance work certifies recovery *deadlines* for adapting controllers via conformal prediction [36]; Bouncer is complementary, certifying a competence floor rather than a recovery time, and adds the secrecy-based mimicry resistance that an observable certificate lacks.

Change detection and OOD monitoring. Bouncer's Tier-B confirmer runs a one-sided CUSUM [16] over its competence signal, inheriting the quickest-detection theory that makes CUSUM minimax-optimal for detection delay under a false-alarm constraint [18], [19], with ARL-to-threshold sizing from sequential analysis [17], [37]. Tier-A tripwires draw on the complementary literature of input-side monitoring: softmax-confidence OOD baselines [11], conformal validity under exchangeability [38], [39], concept-drift detection [13], and representation-level two-sample shift tests [12]. The essential distinction is what is measured. OOD and drift monitors flag when the *input distribution* moves, which is necessary but not sufficient and, fatally, *mimickable*: an adversary can keep covariate statistics in-distribution while degrading control quality. Bouncer instead measures *realized competence* through secret dueling, so the trigger is the controller's actual reward deficit, not a proxy for it. The closest neighbors to this signal are therefore not input monitors but recent *label-free deterioration* tests: the Suitability Filter [40] decides post-deployment fitness from a margin/confidence proxy on the model's own outputs (a $\Delta < \tau$ test); Sequential Harmful Shift Detection Without Labels [41] monitors a proxy-error estimator sequentially with a controlled false-positive rate—structurally our S_{res} plus a one-sided CUSUM; and D3M [42] gives label-free deterioration detection with theoretical false-alarm guarantees, the analogue of our $\sigma_\Delta \to H \to D$ false-alarm sizing. Bouncer differs from all three on axes they share: (i) its per-set dueling estimate is drawn from a *secret, reseeded* sampler, so an adversary who holds the observable proxy in-distribution still cannot game it—the mimicry resistance an observable proxy lacks; (ii) it is a $\sim$336-byte hardware embodiment with a provable bounded-regret trust gate, not a one-shot software deploy/no-deploy decision; and (iii) it makes explicit the *set-locality* (and near-stateless-reward) condition that decides *when* a label-free competence proxy is even attributable to the policy that earned it. Input-distribution monitors are the *weakest* of these alternatives—necessary, cheap, and mimickable—which is why Bouncer relegates them to a first-tier gate (S_{in}) and reserves its trust decision for the competence statistic.

Microarchitectural security and co-tenant attacks. The controllers Bouncer audits are also attack surfaces. Contention and timing channels on caches [43], [44], [10], [45] and occupancy channels [9] leak across cores and VMs in shared clouds [46], while branch predictors [47], [8] and prefetchers [48], [7] expose secret adaptive state that an attacker can read out or steer. A learned controller widens this surface: its policy can be poisoned into amplifying leakage or into co-tenant denial of service. Prior defenses harden specific channels; Bouncer is orthogonal and complementary, bounding the *aggregate* damage any steered controller can do by capping its cost at the vetted fallback. Its mimicry resistance (our Proposition) follows precisely from the secrecy of the set assignment, so an attacker who can observe and game stealthy counter-based detectors [45] still cannot tell which accesses are being scored.

ML-for-systems robustness. Deployed learned components are brittle in exactly the regimes Bouncer guards. Oracle-imitation cache controllers lose accuracy off-distribution [49], learned indexes degrade on irregular data [50], and the Park benchmark documents why RL-for-systems policies behave unlike RL-for-games under noisy, non-stationary workloads [51]; this is the hidden, system-level risk catalogued in [52]. Production systems respond with bespoke guardrails: SLO-clamped colocation [53], OOM-bounded autoscaling [54], and fault-triggered fallback in learned software [55]. Bouncer generalizes these one-off, fault- or threshold-triggered safeguards into a single hardware-cheap mechanism with a *continuous* competence audit and a provable performance floor, applicable to any learned microarchitectural controller rather than one tuned guardrail per deployment.

XIV. Discussion and Limitations

Counterfactual cost. Controllers without clean set-sampling (the memory scheduler) force a model-based $\hat{\Delta}$, which weakens Proposition 1; we scope sampling-friendly controllers first. **Redistributive mimicry and the coverage hole.** A global $\hat{\Delta}$ cannot see an attack that preserves the global mean; per-domain dueling raises detection but does *not* fully close it—a finite leader budget B has a resolution floor $\delta_R^{\min}(B)$, and a slow-bleed adversary that keeps its per-window harm below it evades indefinitely (Proposition 1). We therefore claim mimicry resistance only for declared domains audited at that resolution with an intact secret, and name the sub- resolution coverage hole as an open vulnerability rather than hiding it. **Ground-truth labeling.** "Off-policy" is operationalized as a sustained competence drop beyond a fixed margin, against which we measure latency. **Overhead vs. sensitivity.** The auditor dies if the Tier-B duty cycle is too high; the minimal-overhead operating point—the knee of Figure 7(a)—is reported as the result, not a footnote. **From simulation to silicon.** The controlled quantitative claims are validated on a faithful harness; the real ChampSim L1D module (Section XI-J) shows the mechanism survives a real simulator and bounds a corrupted prefetcher to the safe floor across a 3-trace SPEC suite—a full SPEC/GAP sweep with a timeliness-aware reward (and the production controllers of Table I) is the natural next step. **Reward *localizability*, not just fidelity, is decisive.** The ChampSim runs make this concrete and correct a tempting but wrong fix. A per-set cache-hit reward is a poor proxy for a prefetcher's true (latency/MLP) benefit and over-gates a helpful prefetcher on roms (11% tax)—but we showed (Section XI-J) that swapping to the controller's *own* accuracy$\times$timeliness reward does *not* fix it

(11.2% → 9.7% at fixed window). Set-dueling needs a *set-local* reward, and a prefetcher's benefit is intrinsically de-localized (it fetches a different set than it was triggered on), so the faithful reward is the one that cannot be dualized. The practical levers are a longer epoch (which removes the over-gating empirically) and a set-local signal; the structurally clean case is a controller whose decision and outcome share a set—*replacement*. This is a property of the controller-reward *geometry*, and it sharply scopes where set-dueling competence auditing is faithful.

XV. Conclusion

Bouncer turns "is the learned controller still earning its keep?" into one measurable scalar and bounds the answer's cost—for the controllers a secret per-set audit can faithfully measure. Our organizing result is the *set-locality* condition (Definition 3): set-dueling competence auditing is faithful exactly when a controller's reward shares a set with its decision, which makes cache *replacement* the *set-local* case and characterizes prefetching as the de-localized boundary. Our real-systems study sharpens this into a second condition: set-locality is necessary but not sufficient, because a real cache is set-local yet *stateful*, so the secret reseed that buys mimicry resistance confounds its per-window measurement—a secrecy/measurability tension a (near-)*stateless* reward resolves (Remark 1). For the set-local class: by repurposing the set-dueling substrate into a *secret* competence audit, gating an expensive confirmer behind cheap tripwires, and proving a three-term safety floor (detection, audit-exposure, and false-alarm) tied to the CUSUM constants, then—for declared domains audited at resolution $\delta_R^{\min}(B)$ with the secret intact—Bouncer caps the worst-case *competence* (in the controller's own reward) at its vetted fallback, under benign drift and an adaptive mimicry adversary alike, for ~ 336 bytes and no added datapath latency. The robustness is not free, and not unconditional: we show it is purchased exactly by the secrecy of the dueling sets, quantify the price of losing it, and name the sub-resolution slow-bleed adversary outside the audited domain as a real, un-closed vulnerability.

Artifact Appendix

The full artifact is released: the auditor and environment (`bouncer/`), one script per evaluation phase (`experiments/`), including the transfer-function sensitivity sweep (`exp_e3_sensitivity.py`, Section XI-G), all result JSON and figures, the real ChampSim L1D prefetcher *and* LLC replacement modules (`champsim_plugin/bouncer_pref/`, `bouncer_repl/`), and the paper source. `./run_all.sh` regenerates every synthetic result, figure, and the PDF in ∼4 minutes on a laptop (Python 3.12, numpy/scipy/matplotlib); all RNG is explicitly seeded, so results are deterministic. `champsim_plugin/setup_champsim.sh` clones and builds ChampSim, installs the Bouncer modules, fetches public SPEC CPU2017 traces, and reproduces the prefetcher safety-floor study; `champsim_plugin/run_e1_replacement.sh` reproduces the Section XI-J replacement boundary study (Figure 15) on `623.xalancbmk` (C++17 toolchain + vcpkg). The standalone auditor additionally builds with `c++ -std=c++17 bouncer.cc -o bouncer_selftest`.

References

[1] R. Bera, K. Kanellopoulos, A. V. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, "Pythia: A customizable hardware prefetching framework using online reinforcement learning," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 1121–1137.

[2] A. Jain and C. Lin, "Back to the future: Leveraging belady's algorithm for improved cache replacement," in *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 78–89.

[3] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying deep learning to the cache replacement problem," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 413–425.

[4] E. Ipek, O. Mutlu, J. F. Martínez, and R. Caruana, "Self-optimizing memory controllers: A reinforcement learning approach," in *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, 2008, pp. 39–50.

[5] D. A. Jiménez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, 2001, pp. 197–206.

[6] A. Seznec and P. Michaud, "A case for (partially) TAgged GEometric history length branch prediction," *Journal of Instruction-Level Parallelism (JILP)*, vol. 8, pp. 1–23, 2006.

[7] Y. Guo, A. Zigerelli, Y. Zhang, and J. Yang, "Adversarial prefetch: New cross-core cache side channel attacks," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1458–1473.

[8] D. Evtyushkin, R. Riley, N. B. Abu-Ghazaleh, and D. Ponomarev, "BranchScope: A new side-channel attack on directional branch predictor," in *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2018, pp. 693–707.

[9] A. Shusterman, L. Kang, Y. Haskal, Y. Meltser, P. Mittal, Y. Oren, and Y. Yarom, "Robust website fingerprinting through the cache occupancy channel," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, 2019, pp. 639–656.

[10] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-level cache side-channel attacks are practical," in *2015 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2015, pp. 605–622.

[11] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[12] S. Rabanser, S. Günnemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[14] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive insertion policies for high performance caching," in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2007, pp. 381–391.

[15] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High performance cache replacement using re-reference interval prediction (rrip)," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2010, pp. 60–71.

[16] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[17] D. Siegmund, *Sequential Analysis: Tests and Confidence Intervals*, ser. Springer Series in Statistics. New York: Springer-Verlag, 1985.

[18] G. Lorden, "Procedures for reacting to a change in distribution," *The Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, 1971.

[19] G. V. Moustakides, "Optimal stopping times for detecting changes in distributions," *The Annals of Statistics*, vol. 14, no. 4, pp. 1379–1387, 1986.

[20] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely, and J. Emer, "SHiP: Signature-based hit predictor for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2011, pp. 430–441.

[21] A. Jaleel, W. Hasenplaugh, M. K. Qureshi, J. Sebot, S. C. Steely, and J. Emer, "Adaptive insertion policies for managing shared caches," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. ACM, 2008, pp. 208–219.

[22] V. Seshadri, O. Mutlu, M. A. Kozuch, and T. C. Mowry, "The evicted-address filter: A unified mechanism to address both cache pollution and thrashing," in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*. ACM, 2012, pp. 355–366.

[23] M. K. Qureshi and Y. N. Patt, "Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches," in *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, 2006, pp. 423–432.

[24] S. M. Khan, Y. Tian, and D. A. Jiménez, "Sampling dead block prediction for last-level caches," in *Proceedings of the 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, 2010, pp. 175–186.

[25] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Retrospective: Adaptive insertion policies for high-performance caching," in *ISCA@50 25-Year Retrospective: 1996–2020*, 2023, retrospective on the ISCA 2007 DIP/set-dueling paper.

[26] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[27] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*. AAAI Press, 2018, pp. 2669–2678.

[28] E. Altman, *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

[29] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 2017, pp. 22–31.

[30] Y. Chow, O. Nachum, E. Duenez-Guzman, and M. Ghavamzadeh, "A lyapunov-based approach to safe reinforcement learning," in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018, pp. 8092–8101.

[31] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 908–918.

[32] P. S. Thomas, G. Theocharous, and M. Ghavamzadeh, "High confidence policy improvement," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 2380–2388.

[33] R. Laroche, P. Trichelair, and R. T. des Combes, "Safe policy improvement with baseline bootstrapping," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 3652–3661.

[34] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.

[35] U. Mehmood *et al.*, "The black-box simplex architecture for runtime assurance of autonomous CPS," in *NASA Formal Methods (NFM)*, 2022, arXiv:2102.12981.

[36] Shojaei *et al.*, "Conformal recovery-deadline certificates for runtime assurance of adapting controllers," *arXiv preprint arXiv:2606.25371*, 2026.

[37] A. Tartakovsky, I. Nikiforov, and M. Basseville, *Sequential Analysis: Hypothesis Testing and Changepoint Detection*, ser. Monographs on Statistics and Applied Probability. Boca Raton, FL: Chapman & Hall/CRC, 2014.

[38] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.

[39] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-free predictive inference for regression," *Journal of the American Statistical Association*, vol. 113, no. 523, pp. 1094–1111, 2018.

[40] A. Pouget, M. Yaghini, S. Rabanser, and N. Papernot, "Suitability filter: A statistical framework for classifier evaluation in real-world deployment settings," in *International Conference on Machine Learning (ICML)*, 2025, arXiv:2505.22356.

[41] S. I. Amoukou, T. Bewley, S. Mishra, F. Lecue, D. Magazzeni, and M. Veloso, "Sequential harmful shift detection without labels," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2412.12910.

[42] V. Nguyen, C. Shui, V. Giri, S. Arya, A. Verma, F. Razak, and R. G. Krishnan, "Reliably detecting model failures in deployment without labels," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, d3M; arXiv:2506.05047.

[43] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: The case of AES," in *Topics in Cryptology – CT-RSA 2006, The Cryptographers' Track at the RSA Conference*, ser. Lecture Notes in Computer Science, vol. 3860. Springer, 2006, pp. 1–20.

[44] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack," in *23rd USENIX Security Symposium (USENIX Security 14)*. San Diego, CA: USENIX Association, 2014, pp. 719–732.

[45] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, "Flush+flush: A fast and stealthy cache attack," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, ser. Lecture Notes in Computer Science, vol. 9721. Springer, 2016, pp. 279–299.

[46] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2009, pp. 199–212.

[47] D. Evtyushkin, D. Ponomarev, and N. Abu-Ghazaleh, "Jump over ASLR: Attacking branch predictors to bypass ASLR," in *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–13.

[48] D. Gruss, C. Maurice, A. Fogh, M. Lipp, and S. Mangard, "Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 368–379.

[49] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn, "An imitation learning approach for cache replacement," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 6237–6247.

[50] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, and T. Kraska, "Benchmarking learned indexes," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 14, no. 1, pp. 1–13, 2020.

[51] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, R. Addanki, M. K. Shirkoohi, S. He, V. Nathan, F. Cangialosi, S. B. Venkatakrishnan, W.-H. Weng, S. Han, T. Kraska, and M. Alizadeh, "Park: An open platform for learning-augmented computer systems," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 2490–2502.

[52] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015, pp. 2503–2511.

[53] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 450–462.

[54] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, and J. Wilkes, "Autopilot: Workload autoscaling at Google," in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020, pp. 16:1–16:16.

[55] V. Carbune, T. Coppey, A. Daryin, T. Deselaers, N. Sarda, and J. Yagnik, "Smartchoices: Augmenting software with learned implementations," *arXiv preprint arXiv:2304.13033*, 2023.